

Software Verification and Validation

Part Time Lecturer
Talimul Bari Shreshtho

Islamic University of Technology

May 16, 2026



Table of Contents

1 Introduction

2 Formal Language

Common Strategies in Civil and Mechanical Engineering

- Precise calculations and estimations
 - Forces, stress, load, tolerance, etc.
- Hardware redundancy
 - Build systems stronger than necessary
- Robust design
 - A single fault should not cause catastrophic failure
 - Systems include a margin of error
- Clear subsystem separation
 - Local failures remain isolated
- Proven design patterns
 - Reuse designs known to work reliably

Why is Software Different?

- Software computes **discrete** (non-continuous) functions
 - A single bit flip may completely change behavior
 - Small input changes can cause catastrophic outputs
- Redundancy does not easily solve logical errors
 - Replicating faulty software reproduces the same fault
- Weak subsystem isolation
 - Local failures can propagate through the whole system
- Extremely high logical complexity
 - Large state spaces and complex interactions
- Many engineers are not formally trained in correctness

Key Insight

Software reliability is fundamentally different from hardware reliability.

How Do We Achieve Software Correctness?

Primary Strategy: Testing

- Execute the system under selected conditions
- Observe whether behavior matches expectations

Types of Testing

- **Testing against software errors**
 - Design representative test configurations
 - Verify expected behavior
- **Testing against external faults**
 - Inject memory or communication faults
 - Observe system degradation and fault tolerance

Large companies dedicate massive infrastructure to software testing.

The Fundamental Problem with Testing

- Testing checks behavior only for selected inputs
- Exhaustive testing is usually impossible
- Unknown bugs often remain untested

Important Observation

Testing can show the presence of bugs,
but not their absence.

- If developers do not anticipate a defect:
 - They may never create a test case for it
- Passing tests do not guarantee correctness

Challenges in Software Testing

- Test cases are inherently subjective
- Real-world software behavior is difficult to predict
- Software systems evolve continuously
- Complete branch and scenario coverage is expensive

What Can We Learn From This?

- Testing everything is practically impossible
- Complete verification through testing alone is unrealistic
- Software correctness requires additional techniques:
 - Formal verification
 - Static analysis
 - Code reviews
 - Mathematical reasoning
- Early defect detection is critical

Definition

A formal specification is the expression, in a formal language and at some level of abstraction, of a collection of properties that a system should satisfy.

Key Aspects

- **Formal Language**

- Syntax can be mechanically processed and checked
- Semantics is defined unambiguously using mathematics

Key Elements of Formal Specifications

- **Abstraction**

- Describes the system above the level of source code
- Multiple levels of abstraction are possible

- **Properties**

- Expressed using formal logic
- Have precise and well-defined meaning

- **Satisfaction**

- Ideally verified mechanically
- Uses automated deduction and/or model checking techniques

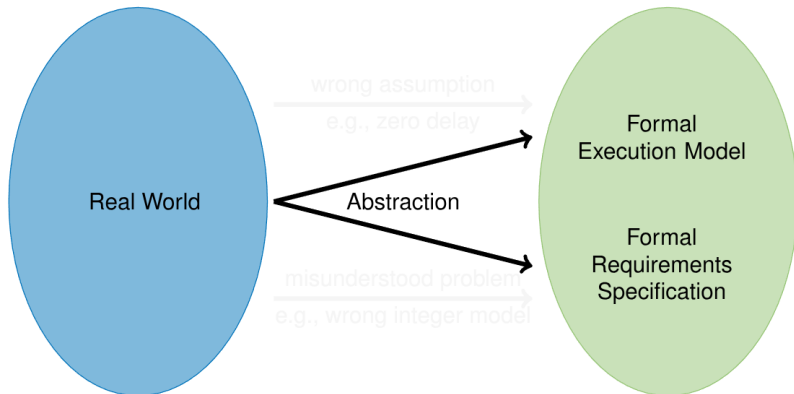
Why Are We Using Alloy in This Course?

- We will use Alloy as a formal specification language
- Alloy helps us:
 - Formally describe system behavior
 - Specify constraints and properties
 - Detect inconsistencies and design flaws
 - Explore counterexamples automatically
- The Alloy Analyzer enables automatic verification of models

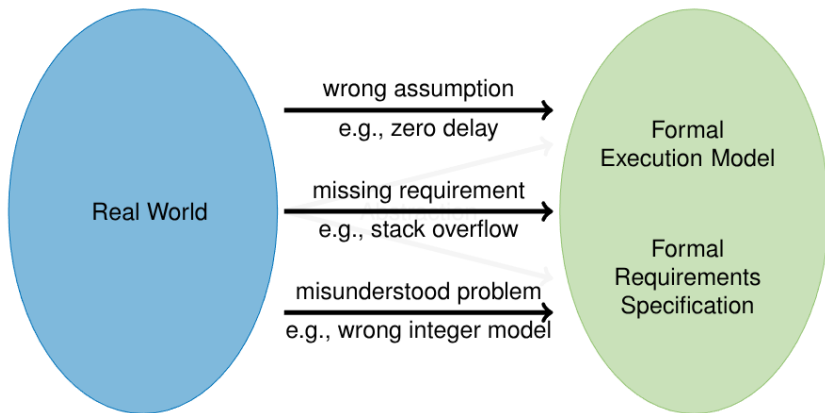
Course Objective

Learn how to formulate, analyze, and verify software systems using Alloy.

Difficulties in Creating Formal Models



Difficulties in Creating Formal Models



Formalizing system requirements is hard
Proving properties of systems can be hard too

Current and Future Trends in Formal Methods

- Formal methods are gradually gaining adoption in industry
- Systems are increasingly being designed with verification in mind
- Integration of:
 - SAT/SMT solvers
 - Theorem provers
 - Semi-automatic verification tools
- Combining:
 - Static analysis
 - Automated verification
 - Theorem proving
- Combining testing with formal verification
- Integration of formal methods into the software development lifecycle

Why is the demand increasing?

- Modern systems require stronger security guarantees
- Security is intrinsically difficult
- Fault-tolerant systems are widely used in safety-critical applications

Challenges

- Security attacks are:
 - Intelligent
 - Coordinated
 - Malicious
- Fault tolerance depends on independent component failures

What is Alloy?

- A lightweight modeling language for software design
- Amenable to fully automatic analysis
- Designed to express complex structural constraints and system behavior
- An intuitive modelling language based on relational logic
- Includes an automatic analyzer based on SAT-solving technology

High-Level Description

- Modelling/programming language level
- Uses:
 - Complex data types
 - Advanced control structures
 - General-purpose programs
- Advantages:
 - Easier to write and understand
 - Faster development
- Limitation:
 - Fully automatic proofs are generally impossible

Low-Level Description

- Machine or hardware level
- System has finitely many states

Advantages

- Automatic verification is, in principle, possible

Disadvantages

- Tedious to program
- Difficult to maintain
- Models can become very large

Definition

- A set is a collection of **distinct** objects
- Objects in a set are drawn from the same domain
- Therefore, sets are **homogeneous**

Examples

- $\{2, 4, 5, 6, \dots\}$ Set of integers
- $\{\text{red}, \text{yellow}, \text{blue}\}$ Set of colors
- $\{\text{true}, \text{false}\}$ Set of Boolean values

Not a Valid Set (under previous domains)

$\{\text{red}, \text{true}, 2\}$

Set Equality and Membership

Value of a Set

- The value of a set is the collection of its members

Set Equality

Two sets A and B are equal iff:

- Every member of A is a member of B
- Every member of B is a member of A

Notation

- $x \in S$: “ x is a member of S ”
- $\emptyset$: empty set

Enumeration

`primaryColors` = {red, yellow, blue}

`Boolean` = {true, false}

`Evens` = {..., -4, -2, 0, 2, 4, ...}

Cardinality

- The cardinality ($\#$) of a finite set is the number of its elements

$$\#\{\text{red, yellow, blue}\} = 3$$

The Problem with “...”

What exactly do we mean by “...” ?

- Does it include all even numbers?
- Does it continue infinitely?
- What is the precise rule?

Key Insight

Formal methods require precision and unambiguous definitions.

REMEMBER: WE WANT TO BE PRECISE

Comprehension Notation

$$\{x : D \mid P(x)\}$$

- Defines a set by selecting elements from a domain D
- Includes exactly those elements that satisfy predicate $P(x)$
- $P(x)$ is a Boolean condition (true/false)

Examples of Set Comprehension

- $\{x : \mathbb{N} \mid x < 10\}$
 - Natural numbers less than 10
- $\{x : \mathbb{Z} \mid (\exists y : \mathbb{Z} \mid x = 2y)\}$
 - Even integers
- $\{x : \mathbb{N} \mid x > x\}$
 - Empty set (no number is greater than itself)

Exercises: Set Comprehension

Try to express using set comprehension notation:

- Odd positive integers
- Squares of integers

Basic Set Properties

- A set that has at least one element
- A set that has exactly one element

Hint

- Choose an appropriate domain ($\mathbb{N}$ or $\mathbb{Z}$)
- Use predicates, existence, and cardinality constraints

Solutions: Set Comprehension

Odd Positive Integers

$$\{x : \mathbb{N} \mid x > 0 \wedge x \bmod 2 = 1\}$$

Squares of Integers

$$\{x : \mathbb{Z} \mid \exists y : \mathbb{Z}, x = y^2\}$$

At Least One Element

$$\{S \mid \exists x : x \in S\}$$

Exactly One Element

$$\{S \mid \exists x : x \in S \wedge \forall y : y \in S \Rightarrow y = x\}$$

Set Operations

Union

$$X \cup Y \equiv \{e : D \mid e \in X \text{ or } e \in Y\}$$

- Combines all elements from both sets
- Example: $\{\text{red}\} \cup \{\text{blue}\} = \{\text{red}, \text{blue}\}$

Intersection

$$X \cap Y \equiv \{e : D \mid e \in X \text{ and } e \in Y\}$$

- Elements common to both sets
- Example: $\{\text{red}, \text{blue}\} \cap \{\text{blue}, \text{yellow}\} = \{\text{blue}\}$

Difference

$$X \setminus Y \equiv \{e : D \mid e \in X \text{ and } e \notin Y\}$$

- Elements in X but not in Y
- Example: $\{\text{red}, \text{yellow}, \text{blue}\} \setminus \{\text{blue}, \text{yellow}\} = \{\text{red}\}$

Subsets and Equality

Subset

- A set X is a subset of Y if every element of X is in Y
- Notation: $X \subseteq Y$

$$\{1, 7, 17, 24\} \subseteq \mathbb{Z}$$

Proper Subset

- $X \subset Y$ means $X \subseteq Y$ and $X \neq Y$

Set Equality

$$A = B \iff (A \subseteq B \wedge B \subseteq A)$$

Definition

- The power set of S , denoted $\text{Pow}(S)$, is the set of all subsets of S

$$\text{Pow}(S) \equiv \{X \mid X \subseteq S\}$$

Example

$$\text{Pow}(\{a, b, c\}) = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$$

Key Note

- $\emptyset \subseteq S$ for any set S
- Therefore $\emptyset \in \text{Pow}(S)$

Set Partitioning

Disjoint Sets

- Two sets are disjoint if they share no elements

Partition

- A partition divides a set S into disjoint subsets (blocks)
- Every element of S belongs to exactly one block

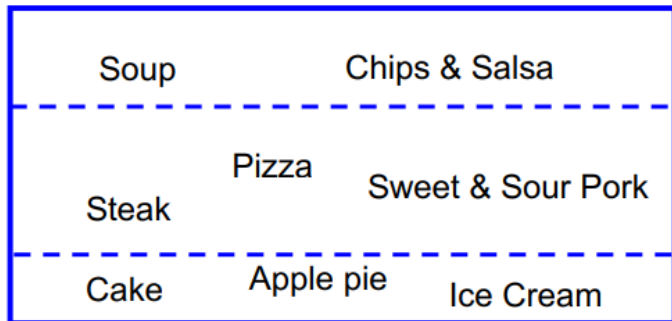


Figure 1: Partition

Cartesian Product

$$A \times B = \{(a, b) \mid a \in A, b \in B\}$$

Example

PrimaryColor $\times$ Boolean

$$= \{(red, true), (red, false), (blue, true), (blue, false), (yellow, true), (yellow, false)\}$$

Binary Relation

- A binary relation R between A and B is an element of $\text{Pow}(A \times B)$, i.e.,

$$R \subseteq A \times B$$

- $R \subseteq A \times B$

Examples

- $\text{Parent} \subseteq \text{Person} \times \text{Person}$
- $\text{Square} \subseteq \mathbb{Z} \times \mathbb{N}$
- $\text{ClassGrades} \subseteq \text{Person} \times \{A, B, C, D, F\}$

$$\text{Parent} = \{(John, Autumn), (John, Sam)\}$$

$$\text{ClassGrades} = \{(Todd, A), (Jane, B)\}$$

Ternary Relation

- A relation $R \subseteq A \times B \times C$

General Case

- An n -ary relation is a subset of $A_1 \times A_2 \times \cdots \times A_n$
- n is called the **arity** of the relation

Key Idea

Relations are the foundation of formal modeling languages like Alloy.

Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`